

indigo*iota*

Platform architecture & tech stack

How we'd build & scale the platform · v1

The strongest answer is also the honest one: the prototype stack is basically the production stack — scaling indigo iota is hardening, not rewriting. The principle throughout is to avoid premature complexity, keep the data layer simple, and spend engineering cycles on extraction quality rather than undifferentiated plumbing.

Guiding principles

One database until it screams. Postgres handles relational data, vectors, full-text, and the graph. Resist polyglot persistence.

Python for the AI core, TypeScript for the app. Don't fight the ecosystems.

Buy the undifferentiated plumbing. OAuth/sync, auth/SSO, LLM observability. The moat is extraction quality and graph-grounded answers — not the Slack connector.

EU-sovereign by construction. It's our wedge with European consultancies.

The stack

Layer	Choice	Why
Web app	Next.js (App Router) + TypeScript + Tailwind + shadcn	Already the demo stack. Hireable, fast; great for the real-time 3D graph (react-force-graph / Three.js) and SSE-driven live updates.
Core API + AI	Python + FastAPI	The extraction pipeline, embeddings, RAG, and agent orchestration live in Python's ecosystem. Async, production-grade.
Datastore	Postgres (EU region) — pgvector (HNSW), tsvector/GIN, JSONB	Hybrid search, relational data, and the graph in one place. Scales further than people assume.
Graph	Stay in Postgres (nodes/edges + recursive CTEs)	Queries are 1–2 hop. No Neo4j until multi-hop graph analytics become a feature — early graph-DB adoption is a premature-complexity trap.
Async / ingestion	Job queue + workers (Redis + RQ/Celery now; Temporal later)	Ingestion is continuous, bursty, multi-step per document; needs durable retryable workflows + a global LLM-concurrency cap.
Connectors	Microsoft Graph, Gmail API, Slack Events API — via a unified layer (Nango, self-hostable)	OAuth, token refresh, delta sync, webhooks — months of plumbing not to hand-roll. Self-hostable keeps it EU-sovereign.
LLM	Model-agnostic gateway (LiteLLM) + per-tenant model policy	Route sovereignty-sensitive tenants to EU-hosted models (Mistral, Azure OpenAI EU, self-hosted); frontier models where permitted. Never hardcode a provider.
Embeddings	Self-hosted (bge / open model, ONNX)	Cheaper, no data egress, sovereignty-clean.

Layer	Choice	Why
Auth	WorkOS (or Clerk)	Consultancies demand SSO/SAML + SCIM directory sync on day one of any enterprise deal.
Files	S3-compatible, EU region (S3 Frankfurt, or MinIO/Scaleway)	Mirror SharePoint files + attachments.
Infra	Managed containers (Fly.io / ECS-Fargate / Scaleway, EU) + managed Postgres (Neon/Supabase EU, RDS Frankfurt)	Kubernetes only when you've outgrown this — not before.
Observability	Sentry + OpenTelemetry + Langfuse (self-hostable)	Token- and quality-bound; you need LLM tracing, evals, and cost visibility early.

The three decisions that actually matter

Multi-tenant data isolation. Product and sales, not just infra. Default: shared Postgres + tenant_id + Row-Level Security. Offer single-tenant / dedicated-DB deployment for the enterprise tier — “your data never co-mingles” is a closing lever, not a cost.

Where the models run. A per-tenant model policy is the technical expression of the EU-sovereignty pitch. Get it wrong and the wedge evaporates.

Buy vs. build the connectors. The real engineering surface area is reliable incremental sync across email/Slack/SharePoint, not the AI. Lean on a unified integration layer.

What we'd deliberately not do early

Neo4j, a separate vector DB (Qdrant/Pinecone), microservices, Kubernetes, Kafka. Each is a plausible “scale” answer and each would slow us down now. They're escape hatches with clear trigger conditions (e.g. move vectors to Qdrant when per-tenant recall/latency on pgvector degrades), not day-one choices.

At a glance

Core API	Python + FastAPI — extraction pipeline, embeddings, RAG, and agent orchestration.
Database	Postgres for everything — relational, vector (pgvector), full-text, and the graph in one store.
Frontend	Next.js + TypeScript — the dashboard, the live 3D graph, and streamed briefings.
Ingestion	A durable job queue — continuous, multi-step sync across email, Slack, and SharePoint.
Ancillary	LLM gateway · observability · OAuth/sync · SSO — bought, not built — so we spend our time on extraction quality.
Uniquely ours	Per-tenant model policy + Row-Level-Security — single-tenant deployments for enterprise — the EU-sovereignty promise that wins European consultancies.
Maturity	Already running the prototype — scaling it is hardening, not a rewrite.